

Dokumentasi Logika Backend

DoleKerjo (pemweb-new)

Platform Pencarian Kerja & Skill Matching

Dokumen ini menjelaskan semua logika backend secara lengkap beserta contoh kode aslinya

Bab 1 — Arsitektur & Tech Stack

1.1 Stack Teknologi yang Dipakai

Komponen	Teknologi	Penjelasan Singkat
Framework	Laravel 11	Framework PHP yang ngatur routing, middleware, database, dsb.
Database	MySQL 8.0	Tempat nyimpan semua data (user, job, lamaran, dll). Jalan via XAMPP.
Login System	Session PHP	Bukan pakai library tambahan. Cukup session bawaan PHP.
AI Matching	OpenRouter API	Pakai AI (Llama 3.3 70B) buat nyocokin CV user sama lowongan.
Baca CV	Python markdown	Script Python yang ubah file PDF/DOCX jadi teks biasa.
Tampilan	Blade + JS biasa	File .blade.php buat HTML, JS vanilla buat interaksi. (Tidak dibahas di sini)

1.2 Cara Kerja Aplikasinya

Secara garis besar, aplikasi ini jalan seperti ini:

- User buka browser** → akses `localhost:8000`

2. **Router Laravel** ngecek URL yang diminta, terus tentuin controller mana yang harus jalan
3. **Middleware** ngecek dulu: user udah login belum? Role-nya apa? Boleh akses halaman ini nggak?
4. **Controller** ambil data dari database (pakai query SQL), olah datanya, terus kirim ke view (Blade)
5. **Blade** render HTML yang akhirnya ditampilkan ke browser user

1.3 Kenapa Pakai Raw SQL?

Sebagian besar query di project ini ditulis langsung sebagai SQL mentah, bukan pakai Eloquent ORM bawaan Laravel. Contohnya:

```
// Ini yang dipake di project – raw SQL langsung $users = DB::select("SELECT * FROM users  
WHERE role = ? ORDER BY created_at DESC", ['user']); // Kalau pakai Eloquent ORM, jadinya  
gini: $users = User::where('role', 'user')->orderBy('created_at', 'desc')->get();
```

Alasannya: dengan raw SQL, kita bisa lihat persis query apa yang dijalankan, gampang bikin JOIN yang kompleks, dan performanya lebih predictable.

Bab 2 — Skema Database

Semua data disimpan di database MySQL bernama `pemweb`. Di bab ini kita bahas satu-satu tabelnya.

2.1 Tabel `users` — Data Semua Akun

Tabel ini nyimpen semua orang yang punya akun di sistem: pencari kerja biasa, admin, dan recruiter (HRD perusahaan).

Kolom	Tipe	Fungsinya
<code>id</code>	Angka, otomatis naik	Nomor unik tiap user (1, 2, 3, ...)
<code>nama</code>	Teks (100 huruf)	Nama lengkap
<code>username</code>	Teks (50 huruf), unik	Username buat login (nggak boleh sama)
<code>email</code>	Teks (100 huruf), unik	Email buat login (nggak boleh sama)
<code>password</code>	Teks (255 huruf)	Password yang udah diacak (di-hash) biar aman
<code>telepon</code>	Teks (20 huruf)	Nomor HP
<code>lokasi</code>	Teks (100 huruf)	Kota/domisili
<code>bio</code>	Teks panjang	Deskripsi singkat tentang diri
<code>foto_profil</code>	Teks (255 huruf)	Alamat file foto di server
<code>tanggal_lahir</code>	Tanggal	Tanggal lahir
<code>gender</code>	Pilihan	laki-laki / perempuan / lainnya
<code>pendidikan</code>	Pilihan	sma / d3 / s1 / s2 / s3
<code>jurusan</code>	Teks (100 huruf)	Jurusan kuliah/sekolah
<code>role</code>	Pilihan	user = pencari kerja, admin = pengelola, recruiter = HRD

is_verified	Angka 0/1	Khusus recruiter: 0 = belum disetujui admin, 1 = sudah
cv	Teks (255 huruf)	Alamat file CV yang diupload
cv_parsed	Teks sangat panjang	Isi CV yang udah diubah jadi teks (buat AI)
portfolio	Teks (255 huruf)	Alamat file portfolio
email_verified	Angka 0/1/2	0 = belum, 2 = nunggu admin, 1 = udah diverifikasi

2.2 Tabel jobs — Data Lowongan Kerja

Nyimpan semua lowongan yang diposting oleh recruiter.

Kolom	Tipe	Fungsinya
id	Angka, otomatis naik	Nomor unik tiap lowongan
nama_posisi	Teks (100 huruf)	Nama jabatan, misal "Frontend Developer"
nama_perusahaan	Teks (100 huruf)	Nama perusahaan yang buka lowongan
lokasi	Teks (100 huruf)	Kota tempat kerja
tipe	Pilihan	full-time / part-time / remote / hybrid / contract / partnership
kategori	Teks (100 huruf)	Kategori kerjaan, misal "Teknologi" atau "Desain"
deskripsi	Teks panjang	Penjelasan tentang pekerjaannya
requirement	Teks panjang	Syarat-syarat yang dibutuhkan
gaji_min	Angka	Gaji paling rendah
gaji_max	Angka	Gaji paling tinggi

<code>recruiter_id</code>	Angka	ID recruiter yang posting (nyambung ke tabel users)
---------------------------	-------	--

2.3 Tabel lamaran — Data Pelamaran

Nyimpan siapa yang ngelamar lowongan apa, dan udah sampai mana prosesnya.

Kolom	Tipe	Fungsinya
<code>user_id</code>	Angka	Siapa yang ngelamar
<code>job_id</code>	Angka	Lowongan mana yang dilamar
<code>status</code>	Pilihan	Proses lamaran (lihat gambar di bawah)
<code>catatan</code>	Teks panjang	Catatan dari recruiter (kalau ada)

Alur status lamaran:

pending (baru dikirim) → review (lagi dicek) → interview (diajak wawancara) → diterima atau ditolak

Satu user cuma bisa ngelamar satu lowongan satu kali aja (ada unique constraint).

2.4 Tabel perusahaan — Profil Perusahaan

Nyimpan data perusahaan milik recruiter. Satu recruiter = satu perusahaan.

Kolom	Fungsinya
<code>recruiter_id</code>	Recruiter yang punya perusahaan ini
<code>nama</code>	Nama perusahaan
<code>logo</code>	File logo perusahaan
<code>deskripsi</code>	Penjelasan tentang perusahaan
<code>lokasi</code>	Alamat/kota perusahaan
<code>website</code>	Website perusahaan
<code>tipe_bisnis</code>	Bidang usaha (misal: "Teknologi", "F&B")

ditemukan_tahun	Tahun perusahaan didirikan
-----------------	----------------------------

2.5 Tabel `skills` dan `user_skills` — Keahlian

`skills` = daftar semua keahlian yang ada di sistem (misal: "JavaScript", "Photoshop", "Excel").

`user_skills` = tabel penghubung antara user dan skill yang dia punya. Soalnya satu user bisa punya banyak skill, dan satu skill bisa dimiliki banyak user (hubungan many-to-many).

```
// Contoh isi user_skills: // user_id=1, skill_id=5, level='mahir' → User 1 jago JavaScript
// user_id=1, skill_id=8, level='pemula' → User 1 pemula di React // user_id=2, skill_id=5,
level='menengah' → User 2 lumayan di JavaScript
```

Level skill ada 3: `pemula`, `menengah`, `mahir`.

2.6 Tabel `reviews` — Ulasan Perusahaan

User bisa kasih review dan rating (bintang 1-5) ke perusahaan. Mirip kayak review di Google Maps.

2.7 Tabel `kategori` — Kategori Lowongan

Daftar kategori yang bisa dipilih admin. Beda dari skill yang pakai ENUM, kategori ini disimpan di tabel terpisah supaya admin bisa tambah/hapus kategori kapan aja tanpa perlu ubah struktur database.

2.8 Tabel `perusahaan_connections` — Koneksi Antar Perusahaan

Nyimpan hubungan antar perusahaan, misalnya partnership atau kerja sama. Strukturnya simpel: perusahaan A terhubung ke perusahaan B, beserta tipe dan catatannya.

2.9 Hubungan Antar Tabel

Ringkasan relasi:

- Satu **recruiter** bisa punya banyak **lowongan**, tapi satu lowongan cuma punya satu recruiter
- Satu **user** bisa ngelamar banyak **lowongan**, tapi satu lowongan cuma dilamar sekali per user
- Satu **recruiter** punya satu **perusahaan** (one-to-one)
- Satu **user** bisa punya banyak **skill**, dan sebaliknya (many-to-many lewat `user_skills`)
- Satu **user** bisa kasih banyak **review** ke berbagai perusahaan
- **Perusahaan** bisa saling terhubung lewat **connections**

Bab 3 — Routing & Middleware

Routing itu kayak "peta" aplikasi — ngatur URL mana yang harus dilayani oleh controller apa. Middleware itu kayak "satpam" — ngecek dulu boleh nggak sih request ini masuk.

3.1 Middleware yang Ada

Ada 3 middleware custom yang didaftarkan di `bootstrap/app.php`:

3.1.1 AuthCheck — "Udah login belum?"

```
// File: app/Http/Middleware/AuthCheck.php public function handle(Request $request, Closure $next) { // Cek apakah ada session user_id if (!session('user_id')) { // Kalau minta JSON (AJAX), balikin 401 if ($request->expectsJson()) { return response()->json(['error' => 'Unauthenticated.'], 401); } // Kalau request biasa, lempar ke halaman login return redirect()->route('login'); } // Kalau ada session-nya, lanjut ke controller return $next($request); }
```

Simpel: kalau nggak ada `user_id` di session, tendang ke halaman login.

3.1.2 AdminOnly — "Khusus admin aja"

```
// File: app/Http/Middleware/AdminOnly.php public function handle(Request $request, Closure $next) { // Harus login DAN role harus 'admin' if (!session('user_id') || session('role') !== 'admin') { if ($request->expectsJson()) { return response()->json(['error' => 'Akses ditolak.'], 403); } abort(403, 'Akses ditolak.');
```

3.1.3 RecruiterOnly — "Khusus recruiter terverifikasi"

```
// File: app/Http/Middleware/RecruiterOnly.php public function handle(Request $request, Closure $next) { // Cek 1: harus login dan role = recruiter if (!session('user_id') || session('role') !== 'recruiter') { // ... tolak akses } // Cek 2: recruiter harus sudah diverifikasi admin // Ini dicek LANGSUNG ke database setiap request $user = DB::selectOne('SELECT is_verified FROM users WHERE id = ?', [$userId]); if (!$user || !$user->is_verified) { session()->flush(); // hapus semua session return redirect()->route('login')->withErrors([ 'login' => 'Akun rekruter Anda belum diverifikasi oleh Admin.', ]); } return $next($request); }
```

Kenapa dicek ulang ke database? Biar kalau admin mencabut verifikasi recruiter, recruiter langsung ke-logout otomatis. Nggak cukup cuma cek session, karena session bisa masih nyimpan data lama.

3.2 Kelompok Route

3.2.1 Route Public (siapa aja boleh akses)

```
// Welcome – redirect otomatis berdasarkan role Route::get('/', function () { if
(session('user_id')) { return match (session('role')) { 'admin' =>
redirect('/admin/dashboard'), 'recruiter' => redirect('/recruiter/dashboard'), default =>
redirect('/home'), }; } return redirect()->route('login'); }); // Auth routes – dibatasi 30
request per menit biar nggak di-spam Route::middleware('throttle:30,1')->group(function () {
Route::get('/login', [AuthController::class, 'showLogin']); Route::post('/login',
[AuthController::class, 'login']); Route::get('/register', [AuthController::class,
'showRegister']); Route::post('/register', [AuthController::class, 'register']);
Route::get('/register/recruiter', [AuthController::class, 'showRegisterRecruiter']);
Route::post('/register/recruiter', [AuthController::class, 'registerRecruiter']); });
```

3.2.2 Route yang Harus Login (auth.check)

```
Route::middleware('auth.check')->group(function () { Route::get('/home', ...); // Halaman
utama Route::get('/profil', ...); // Profil user Route::get('/jobs', ...); // Daftar
lowongan Route::get('/matching', ...); // Skill matching Route::get('/perusahaan', ...); //
Daftar perusahaan // ... dan route-route lainnya });
```

3.2.3 Route Admin (admin.only, prefix: /admin)

```
Route::prefix('admin')->middleware('admin.only')->group(function () {
Route::get('/dashboard', ...); // Dashboard admin Route::get('/jobs', ...); // Kelola
lowongan Route::get('/users', ...); // Kelola user Route::get('/skills', ...); // Kelola
skill & kategori // ... });
```

3.2.4 Route Recruiter (recruiter.only, prefix: /recruiter)

```
Route::prefix('recruiter')->middleware('recruiter.only')->group(function () {
Route::get('/dashboard', ...); // Dashboard recruiter Route::get('/jobs', ...); // Kelola
lowongan sendiri Route::get('/lamaran', ...); // Kelola pelamar Route::get('/profil', ...);
// Kelola profil perusahaan // ... });
```


Bab 4 — AuthController (Sistem Login & Daftar)

File: `app/Http/Controllers/AuthController.php`

Controller ini mengurus semua yang berhubungan sama masuk/keluar akun.

4.1 Proses Login

4.1.1 Validasi Input

Pertama, cek dulu username sama password udah diisi belum:

```
public function login(Request $request) { $request->validate([ 'username' => 'required',  
'password' => 'required', ], [ 'username.required' => 'Email dan password tidak boleh  
kosong.', 'password.required' => 'Email dan password tidak boleh kosong.', ]);
```

4.1.2 Cocokkan Username/Email

User bisa login pakai email ATAU username. Makanya query-nya cek dua kolom:

```
$identifier = trim($request->username); // Cari user yang email-nya ATAU username-nya cocok  
$user = User::where('email', $identifier) ->orWhere('username', $identifier) ->first(); //  
Cek apakah user ketemu DAN password-nya benar if (!$user || !Hash::check($request->password,  
$user->password)) { return back()->withErrors(['login' => 'Email atau password salah.']); }
```

`Hash::check()` itu fungsinya membandingkan password yang diketik user sama password yang udah di-hash (diacak) di database. Kita nggak pernah nyimpan password asli.

4.1.3 Cek Verifikasi Recruiter

```
// Kalau recruiter belum diverifikasi admin, nggak boleh login if ($user->role ===  
'recruiter' && !$user->is_verified) { return back()->withErrors(['login' => 'Akun rekruter  
Anda sedang menunggu verifikasi oleh Admin.']); }
```

4.1.4 Portal User vs Portal Eksekutif

Aplikasi punya 2 "pintu masuk": portal biasa (buat pencari kerja) dan portal eksekutif (buat admin & recruiter). Dicek lewat field `login_mode`:

```
$mode = $request->input('login_mode', 'user'); if ($mode === 'executive') { // Portal  
eksekutif: cuma admin & recruiter yang boleh masuk if (!in_array($user->role, ['admin',  
'recruiter'])) { return back()->withErrors(['login' => 'Akun ini tidak memiliki akses ke  
portal eksekutif.']); } } else { // Portal biasa: cuma user biasa yang boleh masuk if  
($user->role !== 'user') { return back()->withErrors(['login' => 'Akun eksekutif harus masuk  
melalui portal Eksekutif.']); } }
```

4.1.5 Simpan Session & Redirect

Kalau semua lolos, simpan data user ke session terus redirect sesuai role:

```
session([ 'user_id' => $user->id, 'nama' => $user->nama, 'username' => $user->username,
'role' => $user->role, ]); return match($user->role) { 'admin' =>
redirect('/admin/dashboard'), 'recruiter' => redirect('/recruiter/dashboard'), default =>
redirect('/home'), }; }
```

4.2 Daftar Akun User Biasa

4.2.1 Ambil dan Bersihkan Input

```
public function register(Request $request) { $nama = trim($request->input('nama', ''));
$username = trim($request->input('username', '')); $email = trim($request->input('email',
'')); $telepon = trim($request->input('telepon', '')); $lokasi =
trim($request->input('lokasi', '')); $password = $request->input('password', ''); $confirm =
$request->input('confirm_password', ''); $pendidikan = $request->input('pendidikan', '');
$jurusan = trim($request->input('jurusan', ''));
```

`trim()` dipakai buat buang spasi di depan dan belakang, biar " Budi " jadi "Budi".

4.2.2 Validasi Pendidikan

```
// Cuma terima nilai-nilai yang valid $validPendidikan = ['sma', 'd3', 's1', 's2', 's3']; if
(!in_array($pendidikan, $validPendidikan)) { $pendidikan = null; // kalau nggak valid,
anggap nggak diisi }
```

4.2.3 Validasi Manual

```
// Cek field wajib if (!$nama || !$username || !$email || !$password || !$confirm) { return
back()->withErrors(['Harap lengkapi semua kolom yang wajib diisi.'])->withInput(); } // Cek
format email if (!filter_var($email, FILTER_VALIDATE_EMAIL)) { return
back()->withErrors(['Format email tidak valid.'])->withInput(); } // Password minimal 8
karakter if (strlen($password) < 8) { return back()->withErrors(['Password minimal 8
karakter.'])->withInput(); } // Konfirmasi harus sama if ($password !== $confirm) { return
back()->withErrors(['Konfirmasi password tidak cocok.'])->withInput(); }
```

4.2.4 Cek Duplikasi & Simpan

```
// Cek apakah email atau username udah ada yang pakai $existing = DB::selectOne( 'SELECT id
FROM users WHERE email = ? OR username = ?', [$email, $username] ); if ($existing) { return
back()->withErrors(['Email atau username sudah terdaftar.'])->withInput(); } // Simpan ke
database. Password di-hash dulu biar aman! DB::insert(' INSERT INTO users (nama, username,
email, password, telepon, lokasi, pendidikan, jurusan, created_at) VALUES (?, ?, ?, ?, ?, ?,
?, ?, NOW()) ', [$nama, $username, $email, Hash::make($password), $telepon, $lokasi,
```

```
$pendidikan, $jurusan]); // Langsung login-in user yang baru daftar $user =
DB::selectOne('SELECT * FROM users WHERE email = ?', [$email]); session([ 'user_id' =>
$user->id, 'nama' => $user->nama, 'username' => $user->username, 'role' => $user->role, ]);
return redirect()->route('home'); }
```

4.3 Daftar Akun Recruiter

Beda dari daftar user biasa, recruiter harus sekalian isi data perusahaan. Prosesnya dibungkus dalam *transaction* — artinya kalau salah satu gagal, semua dibatalkan (nggak setengah-setengah).

```
public function registerRecruiter(Request $request) { // ... validasi sama kayak register
user biasa ... DB::transaction(function () use ($nama, $username, $email, ...) { // Langkah
1: Buat akun recruiter dulu DB::insert(' INSERT INTO users (nama, username, email, password,
telepon, role, is_verified, created_at) VALUES (?, ?, ?, ?, ?, \'recruiter\', 0, NOW()) ',
[$nama, $username, $email, Hash::make($password), $telepon]); // Catatan: is_verified = 0,
artinya belum disetujui admin // Langkah 2: Ambil ID user yang baru dibuat $user =
DB::selectOne('SELECT id FROM users WHERE email = ?', [$email]); // Langkah 3: Buat data
perusahaan DB::insert(' INSERT INTO perusahaan (recruiter_id, nama, lokasi, website,
tipe_bisnis, ditemukan_tahun, deskripsi, created_at) VALUES (?, ?, ?, ?, ?, ?, ?, NOW()) ',
[$user->id, $nama_perusahaan, $lokasi, $website, $tipe_bisnis, $ditemukan_tahun,
$deskripsi]); }); // Tampilkan halaman sukses return view('auth.regis_recruiter_success');
```

Penting: Recruiter yang baru daftar BELUM bisa login. Dia harus nunggu admin ngeklik "Verifikasi" dulu di panel admin. Ini biar sembarang orang nggak bisa bikin akun recruiter dan posting lowongan palsu.

4.4 Reset Password

```
public function resetPassword(Request $request) { $email = trim($request->input('email',
'')); $password = $request->input('password', ''); $confirm =
$request->input('confirm_password', ''); // Validasi if (strlen($password) < 8) { return
response()->json(['status' => 'error', 'message' => 'Password minimal 8 karakter.']); } if
($password !== $confirm) { return response()->json(['status' => 'error', 'message' =>
'Konfirmasi password tidak cocok.']); } // Cari user, update password $user =
User::where('email', $email)->first(); if (!$user) { return response()->json(['status' =>
'error', 'message' => 'Email tidak terdaftar.']); } $user->password = Hash::make($password);
$user->save(); return response()->json(['status' => 'success', 'message' => 'Password
berhasil diubah. Silakan login.']); }
```

4.5 Cek Email (Anti-tebak)

```
public function checkEmail(Request $request) { // SELALU balas dengan pesan yang sama, nggak
peduli emailnya ada atau nggak // Tujuannya: biar orang jahat nggak bisa nebak email mana
yang terdaftar return response()->json([ 'status' => 'success', 'message' => 'Jika email
terdaftar, instruksi reset password akan diproses.', ]); }
```

4.6 Logout

```
public function logout() { session()->flush(); // hapus semua data session return  
redirect()->route('login'); // balik ke halaman login }
```

Bab 5 — HomeController (Halaman Utama)

File: `app/Http/Controllers/HomeController.php`

Controller ini cuma punya satu method: `index()`. Tugasnya nyiapin data yang ditampilkan di halaman home.

5.1 Ambil Statistik

Buat nampilin angka-angka keren di halaman utama:

```
public function index() { $userId = session('user_id'); // Hitung jumlah perusahaan,
lowongan, dan pencari kerja $totalCompanies = DB::selectOne("SELECT COUNT(*) AS c FROM
perusahaan")->c; $totalJobs = DB::selectOne("SELECT COUNT(*) AS c FROM jobs")->c;
$totalUsers = DB::selectOne("SELECT COUNT(*) AS c FROM users WHERE role = 'user'")->c;
```

5.2 Ambil 4 Lowongan Terbaru

```
// Ambil 4 job paling baru, sekalian hitung berapa yang udah ngelamar $recentJobs =
DB::select(" SELECT j.id, j.nama_posisi, j.nama_perusahaan, j.lokasi, j.tipe, j.kategori,
j.created_at, COUNT(l.id) AS total_pelamar FROM jobs j LEFT JOIN lamaran l ON l.job_id =
j.id GROUP BY j.id ORDER BY j.created_at DESC LIMIT 4 ");
```

`LEFT JOIN` dipakai biar job yang belum ada pelamarnya tetep muncul (`total_pelamar = 0`).

5.3 Ambil 2 Review Terbaru

```
// Ambil review paling baru beserta nama yang ngasih review $recentReviews = DB::select("
SELECT r.rating, r.isi_review, r.nama_perusahaan, u.nama AS reviewer FROM reviews r LEFT
JOIN users u ON u.id = r.user_id ORDER BY r.created_at DESC LIMIT 2 ");
```

5.4 Mapping Tipe Job

```
// Ubah tipe job (misal "full-time") jadi label yang cantik ("Full Time") // dan class CSS
buat warna badge-nya $tipeMap = config('tipe_map'); foreach ($recentJobs as $job) { $map =
$tipeMap[$job->tipe] ?? ['class' => '', 'label' => ucfirst($job->tipe)]; $job->tipe_class =
$map['class']; $job->tipe_label = $map['label']; } return view('home',
compact('totalCompanies', 'totalJobs', 'totalUsers', 'recentJobs', 'recentReviews')); }
```

Bab 6 — JobController (Lowongan Kerja)

File: `app/Http/Controllers/JobController.php`

Ngurus tampilan daftar lowongan dan detail lowongan.

6.1 Daftar Lowongan dengan Filter

6.1.1 Ambil Parameter Filter

User bisa filter lowongan berdasarkan kategori, tipe kerja, lokasi, dan urutan:

```
public function index(Request $request) { $userId = session('user_id'); $sort =
$request->input('sort', 'terbaru'); // urutan $kategori = $request->input('kategori', []);
// array: ["Teknologi", "Desain"] $tipe = $request->input('tipe', []); // array:
["full-time", "remote"] $lokasi = $request->input('lokasi', []); // array: ["Jakarta",
"Bandung"]
```

6.1.2 Tentukan Urutan

```
// Kalau user pilih "gaji", urutkan dari gaji tertinggi // Selain itu, urutkan dari yang
terbaru $orderBy = match($sort) { 'gaji' => 'j.gaji_max DESC', default => 'j.created_at
DESC', };
```

6.1.3 Bangun Query WHERE Secara Dinamis

Ini bagian paling menarik — query SQL-nya dibangun secara dinamis, tergantung filter mana yang dipilih user:

```
$where = []; $binds = []; // Kalau user pilih kategori, tambahkan kondisi IN if
(!empty($kategori)) { $placeholders = implode(',', array_fill(0, count($kategori), '?'));
$where[] = "j.kategori IN ($placeholders)"; $binds = array_merge($binds, $kategori); } //
Hasil kalau pilih 2 kategori: "j.kategori IN (?, ?)" // Kalau user pilih tipe kerja if
(!empty($tipe)) { $placeholders = implode(',', array_fill(0, count($tipe), '?')); $where[] =
"j.tipe IN ($placeholders)"; $binds = array_merge($binds, $tipe); } // Kalau user pilih
lokasi — khusus "remote" beda cara ceknya if (!empty($lokasi)) { $lokasiKondisi = [];
foreach ($lokasi as $l) { if (strtolower($l) === 'remote') { // Remote bukan nama kota, tapi
tipe kerja $lokasiKondisi[] = "j.tipe = 'remote'"; } else { // Kota pakai LIKE biar
fleksibel ("Jakarta" ketemu "DKI Jakarta") $lokasiKondisi[] = "j.lokasi LIKE ?"; $binds[] =
'%' . $l . '%'; } } $where[] = '(' . implode(' OR ', $lokasiKondisi) . ')'; } // Gabung
semua kondisi WHERE $whereSQL = !empty($where) ? 'WHERE ' . implode(' AND ', $where) : '';
```

6.1.4 Jalankan Query Utama

```
// Query ambil semua job + info tambahan $jobs = DB::select(" SELECT j.*, COUNT(l.id) AS
total_pelamar, MAX(CASE WHEN l_user.id IS NOT NULL THEN 1 ELSE 0 END) AS sudah_lamar FROM
```

```
jobs j LEFT JOIN lamaran l ON l.job_id = j.id LEFT JOIN lamaran l_user ON l_user.job_id = j.id AND l_user.user_id = ? {$whereSQL} GROUP BY j.id ORDER BY {$orderBy} ", array_merge([$userId], $binds));
```

Ada 2 kali LEFT JOIN ke tabel lamaran:

- JOIN pertama (l): buat ngitung total pelamar per job
- JOIN kedua (l_user): buat ngecek "user ini udah ngelamar job ini belum?" — hasilnya sudah_lamar = 1 atau 0

6.1.5 Ambil Data Filter

```
// Ambil semua kota unik buat dropdown filter lokasi $locations = DB::select(" SELECT DISTINCT lokasi FROM jobs WHERE lokasi IS NOT NULL AND lokasi != '' ORDER BY lokasi ASC ");
// Ambil kategori dari tabel kategori, urutkan yang paling banyak job-nya $categories = DB::select(" SELECT k.nama, (SELECT COUNT(*) FROM jobs WHERE kategori = k.nama) AS job_count FROM kategori k ORDER BY job_count DESC, k.nama ASC "); return view('jobs', compact('jobs', 'sort', 'kategori', 'tipe', 'lokasi', 'locations', 'categories')); }
```

6.2 Detail Lowongan

Method `show($id)` nampilin detail satu lowongan. Yang menarik: juga nampilin job lain dari perusahaan yang sama.

```
public function show($id) { $userId = session('user_id'); // Ambil detail job + cek apakah user udah ngelamar $job = DB::selectOne(" SELECT j.*, COUNT(l.id) AS total_pelamar, MAX(CASE WHEN l_user.id IS NOT NULL THEN 1 ELSE 0 END) AS sudah_lamar FROM jobs j LEFT JOIN lamaran l ON l.job_id = j.id LEFT JOIN lamaran l_user ON l_user.job_id = j.id AND l_user.user_id = ? WHERE j.id = ? GROUP BY j.id ", [$userId, $id]); if (!$job) abort(404); // kalau job nggak ketemu, tampilkan halaman 404 // Cari info perusahaan yang posting lowongan ini $perusahaan = DB::selectOne(" SELECT p.*, u.nama AS recruiter_nama FROM perusahaan p LEFT JOIN users u ON u.id = p.recruiter_id WHERE p.nama = ? LIMIT 1 ", [$job->nama_perusahaan]); // Cari max 5 lowongan lain dari perusahaan yang sama $relatedJobs = DB::select(" SELECT id, nama_posisi, lokasi, tipe, created_at FROM jobs WHERE nama_perusahaan = ? AND id != ? ORDER BY created_at DESC LIMIT 5 ", [$job->nama_perusahaan, $job->id]); return view('job_detail', compact('job', 'perusahaan', 'relatedJobs')); }
```

6.3 Endpoint JSON

Method `data($id)` sederhana — cuma balikin data job dalam format JSON. Dipakai sama frontend buat nampilin detail job di modal (popup).

```
public function data($id) { $job = DB::selectOne('SELECT * FROM jobs WHERE id = ?', [$id]); if (!$job) return response()->json(['error' => 'Not found'], 404); return response()->json($job); }
```

Bab 7 — ProfilController (Profil Pengguna)

File: `app/Http/Controllers/ProfilController.php`

Ngurus semua yang berhubungan dengan profil user: lihat, edit, upload file, skill, dan hapus akun.

7.1 Ambil Data Profil (JSON)

Method `getData()` dipanggil lewat AJAX. Balikin semua data profil dalam satu paket:

```
public function getData() { $user_id = session('user_id'); // Ambil data user (tanpa password!) $user = DB::selectOne(' SELECT id, nama, username, email, telepon, lokasi, bio, pendidikan, jurusan, tanggal_lahir, gender, foto_profil, cv, portfolio, email_verified, created_at FROM users WHERE id = ? ', [$user_id]); if (!$user) { return response()->json(['status' => 'error', 'message' => 'User tidak ditemukan.'], 404); } // Ambil skill yang dimiliki user $skills = DB::select(' SELECT us.skill_id, s.nama, s.kategori, us.level FROM user_skills us JOIN skills s ON s.id = us.skill_id WHERE us.user_id = ? ', [$user_id]); // Hitung persentase kelengkapan profil $steps = $this->calcCompletion($user, $skills);
```

7.1.1 Cleanup Lamaran Orphan

```
// Hapus lamaran yang job-nya udah dihapus (biar nggak nyangkut) DB::delete(' DELETE l FROM lamaran l LEFT JOIN jobs j ON j.id = l.job_id WHERE l.user_id = ? AND j.id IS NULL ', [$user_id]);
```

Ini perlu karena kadang job dihapus recruiter, tapi lamarannya masih nyangkut di database. Query ini otomatis bersihin.

7.1.2 Ambil Daftar Lamaran User

```
// Ambil semua lamaran user beserta info job-nya $lamaran = DB::select(' SELECT l.status, l.catatan, l.created_at, l.updated_at, j.nama_posisi, j.nama_perusahaan, j.id AS job_id FROM lamaran l JOIN jobs j ON j.id = l.job_id WHERE l.user_id = ? ORDER BY l.created_at DESC ', [$user_id]); // Kirim semua data sebagai JSON return response()->json([ 'status' => 'success', 'user' => $user, 'skills' => $skills, 'lamaran' => $lamaran, 'steps' => $steps, 'completion' => $steps['percent'], ]); }
```

7.2 Update Info Pribadi

```
public function updateInfo(Request $request) { $user_id = session('user_id'); $nama = trim($request->input('nama', '')); $email = trim($request->input('email', '')); $telepon = trim($request->input('telepon', '')); $lokasi = trim($request->input('lokasi', '')); $bio = trim($request->input('bio', '')); // Nama dan email wajib if (!$nama || !$email) { return response()->json(['status' => 'error', 'message' => 'Nama dan email wajib diisi.']); } // Cek: email baru ini udah dipakai orang lain belum? $existing = DB::selectOne( 'SELECT id
```



```
FROM users WHERE email = ? AND id != ?', [$email, $user_id] ); if ($existing) { return
response()->json(['status' => 'error', 'message' => 'Email sudah digunakan akun lain.']); }
// Update di database DB::update(' UPDATE users SET nama=?, email=?, telepon=?, lokasi=?,
bio=?, updated_at=NOW() WHERE id = ? ', [$nama, $email, $telepon, $lokasi, $bio, $user_id]);
// Update juga nama di session (biar di navbar langsung berubah) session(['nama' => $nama]);
return response()->json(['status' => 'success', 'message' => 'Profil berhasil
diperbarui.']); } }
```

7.3 Ganti Password

```
public function updatePassword(Request $request) { $user_id = session('user_id'); $old_pass
= $request->input('old_password', ''); $new_pass = $request->input('new_password', '');
$confirm_pass = $request->input('confirm_password', ''); // ... validasi ... // Cek password
lama dulu – harus benar $user = DB::selectOne('SELECT password FROM users WHERE id = ?',
[$user_id]); if (!Hash::check($old_pass, $user->password)) { return
response()->json(['status' => 'error', 'message' => 'Password saat ini salah.']); } //
Simpan password baru (di-hash) DB::update('UPDATE users SET password = ?, updated_at = NOW()
WHERE id = ?', [ Hash::make($new_pass), $user_id ]); }
```

7.4 Hapus Akun

Ini fitur "hapus akun permanen". Semua data user dihapus bersih. Dibungkus transaction biar kalau gagal di tengah, nggak ada data yang setengah terhapus.

```
public function hapusAkun() { $user_id = session('user_id'); DB::transaction(function () use
($user_id) { $user = DB::selectOne('SELECT role FROM users WHERE id = ?', [$user_id]); if
($user && $user->role === 'recruiter') { // Kalau recruiter: hapus perusahaan, job-jobnya,
lamaran, review $perusahaan = DB::selectOne('SELECT id, nama FROM perusahaan WHERE
recruiter_id = ?', [$user_id]); if ($perusahaan) { DB::delete('DELETE l FROM lamaran l JOIN
jobs j ON l.job_id = j.id WHERE j.recruiter_id = ?', [$user_id]); DB::delete('DELETE FROM
jobs WHERE recruiter_id = ?', [$user_id]); DB::delete('DELETE FROM reviews WHERE
nama_perusahaan = ?', [$perusahaan->nama]); DB::delete('DELETE FROM perusahaan WHERE
recruiter_id = ?', [$user_id]); } } else { // Kalau user biasa: hapus lamaran, skill, review
DB::delete('DELETE FROM lamaran WHERE user_id = ?', [$user_id]); DB::delete('DELETE FROM
user_skills WHERE user_id = ?', [$user_id]); DB::delete('DELETE FROM reviews WHERE user_id =
?', [$user_id]); } // Hapus file-file yang pernah diupload dari disk $userData =
DB::selectOne('SELECT cv, foto_profil, portfolio FROM users WHERE id = ?', [$user_id]); if
($userData) { foreach ([$userData->cv, $userData->foto_profil, $userData->portfolio] as
$file) { if ($file && file_exists(public_path($file))) { @unlink(public_path($file)); //
hapus file dari server } } } // Terakhir: hapus akun user-nya sendiri DB::delete('DELETE
FROM users WHERE id = ?', [$user_id]); }); session()->flush(); // hapus session return
response()->json(['status' => 'success', 'message' => 'Akun berhasil dihapus.']); }
```

7.5 Upload CV

Selain nyimpan file CV, method ini juga coba **baca isi CV** pakai Python biar bisa dipake AI Matching nanti.

```

public function uploadCV(Request $request) { $user_id = session('user_id'); if
(!$request->hasFile('cv')) { return response()->json(['status' => 'error', 'message' =>
'Tidak ada file yang diunggah.']); } $file = $request->file('cv'); $ext =
strtolower($file->getClientOriginalExtension()); // Cuma terima PDF, DOCX, DOC if
(!in_array($ext, ['pdf', 'docx', 'doc'])) { return response()->json(['status' => 'error',
'message' => 'Format file harus PDF, DOCX, atau DOC.']); } // Maksimal 5 MB if
($file->getSize() > 5 * 1024 * 1024) { return response()->json(['status' => 'error',
'message' => 'Ukuran file maksimal 5 MB.']); } // Bikin nama file unik: cv_1_1781622630.pdf
$filename = 'cv_' . $user_id . '_' . time() . '.' . $ext; $dest = public_path('uploads/cv');
if (!is_dir($dest)) mkdir($dest, 0755, true); // bikin foldernya kalau belum ada
$file->move($dest, $filename); // Hapus CV lama kalau ada $old = DB::selectOne('SELECT cv
FROM users WHERE id = ?', [$user_id]); if ($old && $old->cv &&
file_exists(public_path($old->cv))) { @unlink(public_path($old->cv)); } $path =
'uploads/cv/' . $filename; // === BAGIAN PENTING: Parse CV jadi teks === $cvParsed = null;
try { $fullPath = $dest . '/' . $filename; // Jalankan script Python "markdown" buat ubah
PDF/DOCX → teks Markdown $cmd = 'python -m markdown ' . escapeshellarg($fullPath) . '
2>&1'; $output = shell_exec($cmd); if ($output && strlen(trim($output)) > 10) { $cvParsed =
trim($output); } } catch (\Throwable $e) { $cvParsed = null; // kalau gagal parse, nggak
apa-apa } // Simpan path file dan hasil parse ke database DB::update('UPDATE users SET cv =
?, cv_parsed = ?, updated_at = NOW() WHERE id = ?', [$path, $cvParsed, $user_id]); return
response()->json(['status' => 'success', 'message' => 'CV berhasil diunggah!' . ($cvParsed
? ' (AI berhasil membaca CV)' : ''), 'path' => $path, 'parsed' => $cvParsed !== null, []]); }

```

7.6 Upload Foto Profil

Sama kayak upload CV, tapi untuk foto. Bedanya: format file dan batas ukuran.

```

public function uploadPhoto(Request $request) { // ... validasi ... // Format: JPG, JPEG,
PNG, WEBP if (!in_array($ext, ['jpg', 'jpeg', 'png', 'webp'])) { ... } // Maksimal 3 MB if
($file->getSize() > 3 * 1024 * 1024) { ... } // Simpan ke uploads/avatar/ $filename =
'avatar_' . $user_id . '_' . time() . '.' . $ext; // ... pindahkan file, hapus foto lama,
update database ... }

```

7.7 Upload Portfolio

Format: PDF, DOCX, DOC, PPTX, ZIP. Maksimal 10 MB. Disimpan di `uploads/portfolio/`.

7.8 Manajemen Skill User

7.8.1 Lihat Skill yang Belum Dipakai

```

public function skillsList() { $user_id = session('user_id'); // Ambil skill yang BELUM
dimiliki user ini (buat dropdown "tambah skill") $skills = DB::select(' SELECT s.id, s.nama,
s.kategori FROM skills s WHERE s.id NOT IN (SELECT skill_id FROM user_skills WHERE user_id =
?) ORDER BY s.kategori, s.nama ', [$user_id]); return response()->json(['status' =>
'success', 'skills' => $skills]); }

```

7.8.2 Tambah Skill

```
public function addSkill(Request $request) { $user_id = session('user_id'); $skill_id = $request->input('skill_id'); $level = $request->input('level', 'pemula'); // Cek: udah pernah ditambah belum? $exists = DB::selectOne( 'SELECT id FROM user_skills WHERE user_id = ? AND skill_id = ?', [$user_id, $skill_id] ); if ($exists) { return response()->json(['status' => 'error', 'message' => 'Skill sudah ditambahkan.']); } // Validasi level if (!in_array($level, ['pemula', 'menengah', 'mahir'])) { $level = 'pemula'; } DB::insert('INSERT INTO user_skills (user_id, skill_id, level) VALUES (?, ?, ?)', [$user_id, $skill_id, $level]); }
```

7.8.3 Hapus Skill

```
public function removeSkill($skillId) { $user_id = session('user_id'); DB::delete('DELETE FROM user_skills WHERE user_id = ? AND skill_id = ?', [$user_id, $skillId]); }
```

7.9 Verifikasi Email

Verifikasi email di sini nggak kirim email otomatis. User minta verifikasi → admin yang approve manual.

```
public function verifyEmail() { $user_id = session('user_id'); $user = DB::selectOne('SELECT email_verified FROM users WHERE id = ?', [$user_id]); if ($user && $user->email_verified == 1) { return response()->json(['status' => 'error', 'message' => 'Email sudah diverifikasi.']); } if ($user && $user->email_verified == 2) { return response()->json(['status' => 'error', 'message' => 'Permintaan verifikasi sudah dikirim. Menunggu persetujuan admin.']); } // Set ke 2 = pending (nunggu admin approve) DB::update('UPDATE users SET email_verified = 2, updated_at = NOW() WHERE id = ?', [$user_id]); }
```

7.10 Hitung Kelengkapan Profil

Method private ini ngecek 6 hal dan ngitung berapa persen profil user udah lengkap:

```
private function calcCompletion($user, $skills) { $steps = [ ['id' => 'foto', 'label' => 'Upload foto profil', 'done' => !empty($user->foto_profil)], ['id' => 'info', 'label' => 'Lengkapi data pribadi', 'done' => !empty($user->lokasi) && !empty($user->pendidikan)], ['id' => 'bio', 'label' => 'Tulis tentang saya', 'done' => !empty($user->bio)], ['id' => 'cv', 'label' => 'Upload CV', 'done' => !empty($user->cv)], ['id' => 'skill', 'label' => 'Tambah keahlian', 'done' => count($skills) > 0], ['id' => 'email', 'label' => 'Verifikasi email', 'done' => !empty($user->email_verified) && $user->email_verified == 1], ]; $doneCount = collect($steps)->where('done', true)->count(); $total = count($steps); $percent = round(($doneCount / $total) * 100); return [ 'items' => $steps, // daftar langkah + statusnya 'done' => $doneCount, // berapa yang udah selesai 'total' => $total, // total langkah (6) 'percent' => $percent, // persentase (0-100) ]; }
```

Jadi kalau user udah upload foto, isi bio, dan punya skill → 3/6 = 50% lengkap.

Bab 8 — MatchingController (Skill Matching & AI)

File: `app/Http/Controllers/MatchingController.php`

Ini fitur andalan DoleKerjo: nyocokin skill user sama lowongan yang ada. Ada 2 cara: algoritma biasa dan pakai AI.

8.1 Halaman Matching

```
public function index() { $userId = session('user_id'); // Ambil skill user buat di-preload ke form $userSkills = DB::select(" SELECT s.nama, s.kategori FROM user_skills us JOIN skills s ON s.id = us.skill_id WHERE us.user_id = ? ORDER BY s.kategori, s.nama ", [$userId]); // Cek: user punya CV yang udah di-parse nggak? // Kalau punya, tombol "AI Matching" baru aktif $hasCV = DB::selectOne('SELECT cv, cv_parsed FROM users WHERE id = ?', [$userId]); return view('matching', [ 'userSkills' => $userSkills, 'hasCV' => $hasCV && $hasCV->cv && $hasCV->cv_parsed, ]); }
```

8.2 Algoritma Matching Biasa

Method `hasil()` adalah inti dari skill matching. Algoritmanya:

8.2.1 Kumpulkan Semua Skill

```
public function hasil(Request $request) { $userId = session('user_id'); // Skill tambahan dari form input (user bisa ketik skill yang belum ada di profil) $skillInput = $request->input('skills', ''); $extraSkills = array_filter(array_map('trim', explode(',', $skillInput))); // Input "React, Vue, Node" → array ["React", "Vue", "Node"] // Ambil skill dari profil user $profileSkills = DB::select(" SELECT s.nama, s.kategori FROM user_skills us JOIN skills s ON s.id = us.skill_id WHERE us.user_id = ? ", [$userId]); // Gabung keduanya, buang yang duplikat $allSkills = []; $skillCategories = []; $seen = []; foreach ($profileSkills as $ps) { $key = mb_strtolower($ps->nama); // lowercase biar nggak case-sensitive if (!isset($seen[$key])) { $allSkills[] = $ps->nama; $skillCategories[$key] = $ps->kategori; $seen[$key] = true; } } // ... hal yang sama untuk extraSkills ... }
```

8.2.2 Hitung Skor Kecocokan Tiap Lowongan

```
$jobs = DB::select("SELECT * FROM jobs ORDER BY created_at DESC"); $scoredJobs = []; foreach ($jobs as $job) { $score = 0; $matchedSkills = []; // Gabung deskripsi + requirement jadi satu teks $jobText = mb_strtolower($job->deskripsi ?? '') . ' ' . mb_strtolower($job->requirement ?? ''); $jobKategori = mb_strtolower($job->kategori ?? ''); foreach ($allSkills as $sk) { $skLower = mb_strtolower($sk); $matched = false; // ATURAN 1: Kalau kategori skill sama dengan kategori job → +30 poin if (isset($skillCategories[$skLower]) && mb_strtolower($skillCategories[$skLower]) === $jobKategori) { $score += 30; $matched = true; } // ATURAN 2: Kalau nama skill disebut di deskripsi/requirement → +15 poin if (mb_strpos($jobText, $skLower) !== false) { $score += 15; $matched = true; } if ($matched) { $matchedSkills[] = $sk; } } // Normalisasi ke persen (0-100) $maxScore = 30 + (15 * count($allSkills)); $percent = $maxScore > 0 ? min(round(($score / $maxScore) * 100), 100) : 0; if ($percent > 0) { $scoredJobs[] = [ 'id' }
```

```
=> $job->id, 'nama_posisi' => $job->nama_posisi, 'match_percent' => $percent,
'matched_skills' => $matchedSkills, // ... field lainnya ... ]; } }
```

8.2.3 Urutkan dan Ambil Top 10

```
// Urutkan dari skor tertinggi ke terendah usort($scoredJobs, fn($a, $b) =>
$b['match_percent'] <=> $a['match_percent']); $topJobs = array_slice($scoredJobs, 0, 10); //
Skor keseluruhan = rata-rata dari 5 job teratas $topForAvg = array_slice($topJobs, 0, 5);
$matchScore = count($topForAvg) > 0 ? round(array_sum(array_column($topForAvg,
'match_percent')) / count($topForAvg)) : 0;
```

8.2.4 Analisis Kekuatan & Kelemahan

```
// Hitung berapa banyak job yang cocok per skill arsort($skillMatchCount); // urutkan dari
yang paling banyak $strengths = []; // skill yang kuat (cocok 3+ lowongan) $weaknesses = [];
// skill yang lemah (cocok 0 lowongan) foreach ($skillAnalysis as $sa) { if ($sa['matches']
>= 3) { $strengths[] = $sa['name'] . ' - cocok dengan ' . $sa['matches'] . ' lowongan'; }
elseif ($sa['matches'] === 0) { $weaknesses[] = $sa['name'] . ' - belum ada lowongan yang
cocok'; } }
```

8.3 AI Matching (Pakai OpenRouter)

Ini fitur yang pakai AI beneran. Bedanya: AI baca CV user secara keseluruhan dan kasih analisis yang lebih mendalam.

8.3.1 Syarat: Harus Punya CV yang Terbaca

```
public function aiMatch() { $userId = session('user_id'); $user = DB::selectOne(' SELECT
nama, pendidikan, jurusan, cv_parsed, bio FROM users WHERE id = ? ', [$userId]); // Kalau
belum upload CV atau CV gagal dibaca, tolak if (!$user || !$user->cv_parsed) { return
redirect()->route('matching')->withErrors([ 'ai' => 'Upload CV terlebih dahulu di halaman
Profil untuk menggunakan AI Matching.' ]); }
```

8.3.2 Siapkan Data buat AI

```
// Ambil skill user beserta level-nya $skills = DB::select(" SELECT s.nama, us.level FROM
user_skills us JOIN skills s ON s.id = us.skill_id WHERE us.user_id = ? ", [$userId]);
$skillList = array_map(fn($s) => $s->nama . ' (' . $s->level . ')', $skills); // Hasil:
["JavaScript (mahir)", "React (menengah)", "Python (pemula)"] // Ambil max 20 job terbaru
$jobs = DB::select("SELECT * FROM jobs ORDER BY created_at DESC LIMIT " .
config('openrouter.max_jobs', 20));
```

8.3.3 Bangun Prompt buat AI

```
private function buildPrompt($user, $skillList, $cvText, $jobTexts) { // System message:
kasih tau AI peran dan aturannya $systemMsg = <<nama> Education: {$user->pendidikan}
{$user->jurusan} Bio: {$user->bio} Skills: {$skillsStr} CV CONTENT (parsed markdown):
{$cvText} // ← isi CV yang udah di-parse, max 4000 karakter AVAILABLE JOBS: {$jobTexts} //
← daftar lowongan, max 20 Return JSON array with jobs scoring 30+, max 10 jobs. Format: [ {
"job_id": , "score": <0-100>, "reasoning": "<2-3 kalimat dalam Bahasa Indonesia>",
"matched_skills": ["skill1", "skill2"], "gaps": ["gap1", "gap2"] } ] USER; return [ ['role'
=> 'system', 'content' => $systemMsg], ['role' => 'user', 'content' => $userMsg], ]; }
```

8.3.4 Panggil API OpenRouter

```
private function callOpenRouter(array $messages): ?string { $apiKey =
config('openrouter.api_key'); // Model utama + 2 model cadangan kalau kena rate limit
$models = [ config('openrouter.model'), 'meta-llama/llama-3.3-70b-instruct:free',
'nousresearch/hermes-3-llama-3.1-405b:free', ]; // Coba satu per satu. Kalau
gagal/rate-limit, coba model berikutnya foreach ($models as $model) { $result =
$this->doApiCall($apiKey, $model, $messages); if ($result !== null) { return $result; //
berhasil! } usleep(500000); // tunggu 0.5 detik sebelum coba lagi } return null; // semua
model gagal }
```

8.3.5 Detail Pemanggilan API

```
private function doApiCall(string $apiKey, string $model, array $messages): ?string {
$payload = [ 'model' => $model, 'messages' => $messages, 'temperature' => 0.2, // rendah =
jawaban lebih konsisten 'max_tokens' => 2000, ]; // Kirim request ke OpenRouter pakai CURL
$sch = curl_init(config('openrouter.base_url')); curl_setopt_array($sch, [ CURLOPT_POST =>
true, CURLOPT_POSTFIELDS => json_encode($payload), CURLOPT_RETURNTRANSFER => true,
CURLOPT_TIMEOUT => 60, CURLOPT_HTTPHEADER => [ 'Content-Type: application/json',
'Authorization: Bearer ' . $apiKey, 'HTTP-Referer: http://localhost:8000', 'X-Title:
DoleKerjo AI Matching', ], ]); $response = curl_exec($sch); $statusCode = curl_getinfo($sch,
CURLINFO_HTTP_CODE); curl_close($sch); if ($statusCode === 429) return null; // rate limit →
coba model lain if ($statusCode !== 200) return null; // error lain $data =
json_decode($response, true); return $data['choices'][0]['message']['content'] ?? null; }
```

8.3.6 Parse Jawaban AI

```
private function parseAIResponse(string $raw, array $jobs): array { // Bersihin kalau AI-nya
ngirim markdown code block $clean = preg_replace('/`(`(?:json)?\s*/i', '', $raw); // Coba
parse JSON-nya $parsed = json_decode($clean, true); if (!is_array($parsed)) { // Kalau
gagal, coba ekstrak pakai regex if (preg_match('/\[.*\]/s', $clean, $matches)) { $parsed =
json_decode($matches[0], true); } } if (!is_array($parsed)) return []; // Cocokkan job_id
dari AI dengan data job yang sebenarnya $jobMap = []; foreach ($jobs as $j) {
$jobMap[$j->id] = $j; } $results = []; foreach ($parsed as $item) { $jobId = $item['job_id']
?? null; if (!$jobId || !isset($jobMap[$jobId])) continue; $j = $jobMap[$jobId]; $results[]
= [ 'id' => $j->id, 'nama_posisi' => $j->nama_posisi, 'match_percent' =>
min(max(intval($item['score']) ?? 0, 0), 100), 'reasoning' => $item['reasoning'] ?? '',
'matched_skills' => $item['matched_skills'] ?? [], 'gaps' => $item['gaps'] ?? [], ]; } //
Urutkan dan ambil top 10 usort($results, fn($a, $b) => $b['match_percent'] <=>
$a['match_percent']); return array_slice($results, 0, 10); }
```

Bab 9 — PerusahaanController (Halaman Perusahaan)

File: `app/Http/Controllers/PerusahaanController.php`

Ngurus halaman-halaman perusahaan: daftar, detail, lamar, review, dan koneksi.

9.1 Daftar Semua Perusahaan

```
public function index() { // Ambil semua perusahaan + statistik per perusahaan $perusahaan =
DB::select(" SELECT p.*, u.nama AS recruiter_nama, COUNT(DISTINCT j.id) AS total_jobs, //
jumlah lowongan ROUND(AVG(r.rating), 1) AS avg_rating, // rata-rata bintang COUNT(DISTINCT
r.id) AS total_reviews // jumlah review FROM perusahaan p LEFT JOIN users u ON u.id =
p.recruiter_id LEFT JOIN jobs j ON j.nama_perusahaan = p.nama LEFT JOIN reviews r ON
r.nama_perusahaan = p.nama GROUP BY p.id ORDER BY p.nama ASC "); return
view('perusahaan.index', [ 'perusahaan' => $perusahaan, 'role' => session('role', 'user'),
'user_id' => session('user_id'), ]); }
```

9.2 Lamar Pekerjaan

```
public function applyJob(Request $request) { // Cuma user biasa yang bisa ngelamar if
(session('role') !== 'user') return response()->json(['error' => 'Hanya user yang bisa
melamar'], 403); $jobId = $request->input('job_id'); $userId = session('user_id'); // Cek
job-nya ada nggak $job = DB::selectOne("SELECT * FROM jobs WHERE id = ?", [$jobId]); if
(!$job) return response()->json(['error' => 'Lowongan tidak ditemukan'], 404); // Cek: udah
pernah ngelamar sebelumnya? $existing = DB::selectOne( "SELECT id FROM lamaran WHERE user_id
= ? AND job_id = ?", [$userId, $jobId] ); if ($existing) return response()->json(['error' =>
'Anda sudah melamar pekerjaan ini'], 400); // Belum pernah → insert lamaran baru, status
awal "pending" DB::insert( "INSERT INTO lamaran (user_id, job_id, status) VALUES (?, ?,
'pending')", [$userId, $jobId] ); return response()->json(['success' => true, 'message' =>
'Lamaran berhasil dikirim!']); }
```

9.3 Kasih Review Perusahaan

```
public function storeReview(Request $request) { $userId = session('user_id'); $namaPrsh =
$request->input('nama_perusahaan'); $rating = (int) $request->input('rating'); $posisi =
$request->input('posisi_user') ?: null; $isi = $request->input('isi_review'); // Validasi if
($rating < 1 || $rating > 5) return response()->json(['error' => 'Rating harus 1-5'], 400);
if (!$isi || strlen(trim($isi)) < 3) return response()->json(['error' => 'Review minimal 3
karakter'], 400); // Kalau udah pernah review perusahaan ini → UPDATE yang lama $existing =
DB::selectOne( "SELECT id FROM reviews WHERE user_id = ? AND nama_perusahaan = ?", [$userId,
$namaPrsh] ); if ($existing) { DB::update("UPDATE reviews SET rating=?, posisi_user=?,
isi_review=? WHERE id=?", [$rating, $posisi, $isi, $existing->id]); } else {
DB::insert("INSERT INTO reviews (user_id, nama_perusahaan, posisi_user, isi_review, rating)
VALUES (?, ?, ?, ?, ?)", [$userId, $namaPrsh, $posisi, $isi, $rating]); } // Balikin rating
terbaru perusahaan $stats = DB::selectOne( "SELECT COUNT(*) AS total, ROUND(AVG(rating),1)
AS avg_rating FROM reviews WHERE nama_perusahaan = ?", [$namaPrsh]); return
response()->json([ 'success' => true, 'avg_rating' => $stats->avg_rating, 'total' =>
$stats->total, ]); }
```

9.4 Lowongan Perusahaan (Beda Role, Beda Hasil)

```
public function getLamaran($id) { $p = DB::selectOne("SELECT * FROM perusahaan WHERE id = ?", [$id]); $role = session('role', 'user'); $userId = session('user_id'); if ($role === 'recruiter') { // Recruiter: cuma lihat job miliknya sendiri $jobs = DB::select(" SELECT j.*, COUNT(l.id) AS total_lamaran FROM jobs j LEFT JOIN lamaran l ON l.job_id = j.id WHERE j.nama_perusahaan = ? AND j.recruiter_id = ? GROUP BY j.id ", [$p->nama, $userId]); } else { // User biasa: lihat semua job + cek udah ngelamar atau belum $jobs = DB::select(" SELECT j.*, COUNT(l_all.id) AS total_lamaran, MAX(CASE WHEN l_user.id IS NOT NULL THEN 1 ELSE 0 END) AS sudah_lamar FROM jobs j LEFT JOIN lamaran l_all ON l_all.job_id = j.id LEFT JOIN lamaran l_user ON l_user.job_id = j.id AND l_user.user_id = ? WHERE j.nama_perusahaan = ? GROUP BY j.id ", [$userId, $p->nama]); } }
```


Bab 10 — AdminController (Panel Admin)

File: `app/Http/Controllers/AdminController.php`

Ngurus dashboard dan pengaturan yang cuma bisa diakses admin.

10.1 Dashboard Admin

```
public function dashboard() { // Hitung semua statistik penting $totalUsers =
DB::selectOne("SELECT COUNT(*) AS c FROM users WHERE role = 'user'")->c; $totalRecruiters =
DB::selectOne("SELECT COUNT(*) AS c FROM users WHERE role = 'recruiter'")->c; $totalJobs =
DB::selectOne("SELECT COUNT(*) AS c FROM jobs")->c; $totalLamaran = DB::selectOne("SELECT
COUNT(*) AS c FROM lamaran")->c; $totalSkills = DB::selectOne("SELECT COUNT(*) AS c FROM
skills")->c; $totalPerusahaan = DB::selectOne("SELECT COUNT(*) AS c FROM perusahaan")->c; //
Berapa recruiter yang masih nunggu verifikasi $pendingRecruiters = DB::selectOne( "SELECT
COUNT(*) AS c FROM users WHERE role = 'recruiter' AND is_verified = 0" )->c; // Hitung
lamaran per status (pending berapa, interview berapa, dst) $lamaranStats =
DB::select("SELECT status, COUNT(*) AS c FROM lamaran GROUP BY status"); // 5 user dan 5 job
terbaru $recentUsers = DB::select("SELECT * FROM users ORDER BY created_at DESC LIMIT 5");
$recentJobs = DB::select("SELECT * FROM jobs ORDER BY created_at DESC LIMIT 5"); return
view('admin.dashboard', compact(...)); }
```

10.2 Kelola Lowongan

10.2.1 Daftar Jobs (dengan Sorting)

```
public function jobs(Request $request) { $sort = $request->get('sort', 'id'); $dir =
strtolower($request->get('dir', 'desc')) === 'asc' ? 'asc' : 'desc'; // Whitelist kolom yang
boleh di-sort (biar nggak bisa inject SQL) $allowed = ['id', 'created_at', 'nama_posisi',
'nama_perusahaan', 'tipe']; if (!in_array($sort, $allowed)) $sort = 'id'; $jobs =
DB::select(" SELECT j.*, u.nama AS recruiter_nama FROM jobs j LEFT JOIN users u ON
j.recruiter_id = u.id ORDER BY j.{ $sort } { $dir } "); }
```

10.2.2 Hapus Job

```
public function deleteJob($id) { // Hapus lamaran dulu, baru hapus job-nya // (karena kalau
langsung hapus job, lamaran bisa jadi orphan) DB::delete('DELETE FROM lamaran WHERE job_id =
?', [$id]); DB::delete('DELETE FROM jobs WHERE id = ?', [$id]); return
redirect('/admin/jobs')->with('success', 'Lowongan berhasil dihapus.');
```

10.3 Kelola User

10.3.1 Daftar Users (dengan Search, Filter, Paginasi)

```
public function users(Request $request) { $search = trim($request->get('search', '')); $role
= $request->get('role', ''); $sort = $request->get('sort', 'id'); $dir =
```

```
strtolower($request->get('dir', 'desc')) === 'asc' ? 'asc' : 'desc'; $page = max(1,
intval($request->get('page', 1))); $perPage = 10; // 10 user per halaman // Bangun kondisi
WHERE dinamis $params = []; $where = []; if ($search !== '') { // Cari berdasarkan nama,
username, atau email $where[] = '(nama LIKE ? OR username LIKE ? OR email LIKE ?)';
$params[] = "%{$search}%"; $params[] = "%{$search}%"; $params[] = "%{$search}%"; } if ($role
!== '' && in_array($role, ['admin', 'recruiter', 'user'])) { $where[] = 'role = ?';
$params[] = $role; } // Hitung total data (buat paginasi) $whereClause = count($where) > 0 ?
'WHERE ' . implode(' AND ', $where) : ''; $total = DB::selectOne("SELECT COUNT(*) AS c FROM
users {$whereClause}", $params)->c; $totalPages = (int) ceil($total / $perPage); $offset =
($page - 1) * $perPage; // Ambil data halaman ini $users = DB::select(" SELECT * FROM users
{$whereClause} ORDER BY {$sort} {$dir} LIMIT {$perPage} OFFSET {$offset} ", $params); }
```

10.4 Verifikasi Recruiter

```
public function verifyRecruiter($id) { // Sempel: set is_verified = 1, recruiter langsung
bisa login DB::update("UPDATE users SET is_verified = 1 WHERE id = ? AND role =
'recruiter'", [$id]); return redirect()->back()->with('success', 'Recruiter berhasil
diverifikasi.');
```

```
public function verifyUserEmail($id) { // Admin approve permintaan
verifikasi email user DB::update("UPDATE users SET email_verified = 1 WHERE id = ?", [$id]);
return redirect()->back()->with('success', 'Email user berhasil diverifikasi.');
```

10.5 Hapus User

```
public function deleteUser($id) { // Admin nggak bisa hapus akun sendiri (biar nggak nggak
sengaja) if ($id == session('user_id')) { return redirect()->back()->with('error', 'Tidak
dapat menghapus akun sendiri.');
```

```
} // Proses hapus sama kayak di ProfilController (Bab 7.4)
DB::transaction(function () use ($id) { // ... hapus data terkait berdasarkan role ... //
... hapus file upload ... DB::delete('DELETE FROM users WHERE id = ?', [$id]); }); }
```

10.6 Kelola Skill & Kategori

```
// Tambah skill baru public function addSkill(Request $request) { $request->validate([
'nama' => 'required|string|max:100', 'kategori' => 'required|string|max:100', ]); try {
DB::insert("INSERT INTO skills (nama, kategori) VALUES (?, ?)", [$request->nama,
$request->kategori]); return redirect('/admin/skills')->with('success', 'Skill berhasil
ditambahkan.');
```

```
} catch (\Exception $e) { return redirect('/admin/skills')->with('error',
'Skill sudah ada atau gagal ditambahkan.');
```

```
} } // Hapus kategori – tapi cek dulu, masih ada
job yang pakai nggak? public function deleteKategori($id) { $kat = DB::selectOne("SELECT
nama FROM kategori WHERE id = ?", [$id]); if ($kat) { $jobCount = DB::selectOne( "SELECT
COUNT(*) AS c FROM jobs WHERE kategori = ?", [$kat->nama] )->c; if ($jobCount > 0) { return
redirect('/admin/skills')->with('error', "Tidak dapat menghapus kategori \"{$kat->nama}\"
karena masih digunakan oleh {$jobCount} lowongan."); } } DB::delete("DELETE FROM kategori
WHERE id = ?", [$id]); }
```

Bab 11 — RecruiterController (Portal Recruiter)

File: `app/Http/Controllers/RecruiterController.php`

Ngurus semua yang bisa dilakukan recruiter: posting lowongan, kelola pelamar, dan profil perusahaan.

11.1 Dashboard Recruiter

```
public function dashboard() { $uid = $this->userId(); // = session('user_id') // Statistik
lowongan milik recruiter ini $totalJobs = DB::selectOne( "SELECT COUNT(*) AS c FROM jobs
WHERE recruiter_id = ?", [$uid] )->c; // Total semua lamaran yang masuk ke job-job recruiter
ini $totalLamaran = DB::selectOne(" SELECT COUNT(*) AS c FROM lamaran l JOIN jobs j ON
l.job_id = j.id WHERE j.recruiter_id = ? ", [$uid])->c; // Hitung per status $pendingLamaran
= DB::selectOne(" SELECT COUNT(*) AS c FROM lamaran l JOIN jobs j ON l.job_id = j.id WHERE
j.recruiter_id = ? AND l.status = 'pending' ", [$uid])->c; // ... sama untuk interview,
diterima, ditolak ... // Rating perusahaan recruiter ini $companyRating = DB::selectOne("
SELECT COUNT(*) AS total, ROUND(AVG(rating), 1) AS avg_rating FROM reviews r JOIN perusahaan
p ON p.nama = r.nama_perusahaan WHERE p.recruiter_id = ? ", [$uid]); // 8 pelamar terbaru
dan 5 job terbaru $recentApplicants = DB::select(" SELECT l.id, l.status, u.nama, u.email,
j.nama_posisi FROM lamaran l JOIN jobs j ON l.job_id = j.id JOIN users u ON l.user_id = u.id
WHERE j.recruiter_id = ? ORDER BY l.created_at DESC LIMIT 8 ", [$uid]); }
```

11.2 Posting Lowongan Baru

```
public function storeJob(Request $request) { $uid = $this->userId(); // Validasi input
$request->validate([ 'nama_posisi' => 'required|string|max:100', 'nama_perusahaan' =>
'required|string|max:100', 'tipe' =>
'required|in:full-time,part-time,remote,hybrid,contract,partnership', 'kategori' =>
'required|string|max:100', 'gaji_min' => 'nullable|integer|min:0', 'gaji_max' =>
'nullable|integer|min:0', ]); // Simpan ke database DB::insert(" INSERT INTO jobs
(nama_posisi, nama_perusahaan, lokasi, tipe, kategori, deskripsi, requirement, gaji_min,
gaji_max, recruiter_id) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?) ", [ $request->nama_posisi,
$request->nama_perusahaan, $request->lokasi, $request->tipe, $request->kategori,
$request->deskripsi, $request->requirement, $request->gaji_min ?: null, $request->gaji_max
?: null, $uid, ]); }
```

11.3 Edit Lowongan (dengan Keamanan)

```
public function updateJob(Request $request, $id) { $uid = $this->userId(); // PENTING: Cek
dulu job ini punya siapa // Biar recruiter A nggak bisa edit job punya recruiter B $job =
DB::selectOne( "SELECT id FROM jobs WHERE id = ? AND recruiter_id = ?", [$id, $uid] ); if
(!$job) abort(403, 'Lowongan tidak ditemukan.');// ... validasi dan update ... DB::update("
UPDATE jobs SET nama_posisi=?, nama_perusahaan=?, lokasi=?, tipe=?, kategori=?, deskripsi=?,
requirement=?, gaji_min=?, gaji_max=? WHERE id = ? AND recruiter_id = ? ", [..., $id,
$uid]); }
```

11.4 Kelola Pelamar

11.4.1 Daftar Pelamar (dengan Filter & Search)

```
public function lamaran(Request $request) { $uid = $this->userId(); $status =
$request->get('status', ''); $search = trim($request->get('search', '')); $page = max(1,
intval($request->get('page', 1))); $perPage = 10; $params = [$uid]; $where =
['j.recruiter_id = ?']; // cuma lamaran ke job milik recruiter ini // Filter berdasarkan
status if ($status !== '' && in_array($status,
['pending','review','interview','diterima','ditolak'])) { $where[] = 'l.status = ?';
$params[] = $status; } // Cari berdasarkan nama pelamar atau posisi if ($search !== '') {
$where[] = '(u.nama LIKE ? OR u.email LIKE ? OR j.nama_posisi LIKE ?)'; $params[] =
"%{$search}%"; $params[] = "%{$search}%"; $params[] = "%{$search}%"; } // Ambil data pelamar
+ info user + info job $applicants = DB::select(" SELECT l.id, l.status, l.catatan,
l.created_at, l.updated_at, u.nama, u.email, u.telepon, u.foto_profil, u.cv, j.nama_posisi,
j.id AS job_id FROM lamaran l JOIN jobs j ON l.job_id = j.id JOIN users u ON l.user_id =
u.id WHERE ... ORDER BY l.created_at DESC LIMIT {$perPage} OFFSET {$offset} ", $params); //
Hitung jumlah per status (buat badge di tombol filter) $statusCounts = DB::select(" SELECT
l.status, COUNT(*) AS c FROM lamaran l JOIN jobs j ON l.job_id = j.id WHERE j.recruiter_id =
? GROUP BY l.status ", [$uid]); }
```

11.4.2 Ubah Status Lamaran

```
public function updateStatus(Request $request, $id) { $uid = $this->userId();
$request->validate(['status' => 'required|in:pending,review,interview,diterima,ditolak']);
// Cek: lamaran ini memang milik job recruiter yang lagi login? $exists = DB::selectOne("
SELECT l.id FROM lamaran l JOIN jobs j ON l.job_id = j.id WHERE l.id = ? AND j.recruiter_id
= ? ", [$id, $uid]); if (!$exists) abort(403, 'Lamaran tidak ditemukan.');
```

```
// Update status
+ catatan DB::update("UPDATE lamaran SET status = ?, catatan = ? WHERE id = ?", [
$request->status, $request->catatan, $id ]); return redirect()->back()->with('success',
'Status lamaran berhasil diperbarui.');
```

11.5 Profil Perusahaan

```
public function updateProfil(Request $request) { $uid = $this->userId();
$request->validate([ 'nama' => 'required|string|max:100', 'lokasi' =>
'nullable|string|max:100', 'website' => 'nullable|string|max:255', 'tipe_bisnis' =>
'nullable|string|max:100', 'ditemukan_tahun' => 'nullable|integer|min:1800|max:2100',
'deskripsi' => 'nullable|string', ]); // Cek: perusahaan udah ada atau belum? $existing =
DB::selectOne("SELECT id FROM perusahaan WHERE recruiter_id = ?", [$uid]); if ($existing) {
// Udah ada → UPDATE DB::update("UPDATE perusahaan SET nama=?, lokasi=?, website=?, ...
WHERE recruiter_id = ?", [..., $uid]); } else { // Belum ada → INSERT baru
DB::insert("INSERT INTO perusahaan (recruiter_id, nama, ...) VALUES (?, ?, ...)", [$uid,
...]); } }
```

Bab 12 — Models

Project ini cuma punya 2 model Eloquent. Tapi jarang dipakai karena kebanyakan query pakai raw SQL.

12.1 Model User

```
// File: app/Models/User.php namespace App\Models; use Illuminate\Foundation\Auth\User as Authenticatable; class User extends Authenticatable { // Field yang boleh diisi massal (pakai create() atau fill()) protected $fillable = [ 'nama', 'username', 'email', 'password', 'telepon', 'lokasi', 'bio', 'foto_profil', 'tanggal_lahir', 'gender', 'pendidikan', 'jurusan', 'role', ]; // Field yang disembunyikan kalau model di-serialize ke JSON protected $hidden = ['password']; } // Cara pakai (di AuthController): $user = User::where('email', $identifier)->first(); // Ini sama aja dengan: DB::selectOne("SELECT * FROM users WHERE email = ?", [$identifier])
```

12.2 Model Perusahaan

```
// File: app/Models/Perusahaan.php namespace App\Models; use Illuminate\Database\Eloquent\Model; class Perusahaan extends Model { // Nama tabel harus ditulis eksplisit // (kalau nggak, Laravel bakal cari tabel "perusahaans" – salah) protected $table = 'perusahaan'; protected $fillable = [ 'recruiter_id', 'nama', 'logo', 'deskripsi', 'lokasi', 'website', 'tipe_bisnis', 'ditemukan_tahun', ]; }
```

Bab 13 — Konfigurasi

13.1 File `config/tipe_map.php`

File ini menyimpan "kamus" tipe lowongan. Dipakai di banyak controller biar label dan warna badge konsisten.

```
return [ 'full-time' => ['class' => '', 'label' => 'Full Time'], 'part-time' => ['class' => 'parttime', 'label' => 'Part Time'], 'remote' => ['class' => 'remote', 'label' => 'Remote'], 'hybrid' => ['class' => 'hybrid', 'label' => 'Hybrid'], 'contract' => ['class' => 'contract', 'label' => 'Contract'], 'partnership' => ['class' => 'partner', 'label' => 'Partnership'], ]; // Cara pakai di controller: $tipeMap = config('tipe_map'); $map = $tipeMap[$job->tipe] ?? ['class' => '', 'label' => ucfirst($job->tipe)]; $job->tipe_label = $map['label']; // "full-time" → "Full Time" $job->tipe_class = $map['class']; // "remote" → "remote" (CSS class)
```

13.2 File `config/openrouter.php`

Konfigurasi buat AI Matching:

```
return [ 'api_key' => env('OPENROUTER_API_KEY', ''), // dari file .env 'base_url' => 'https://openrouter.ai/api/v1/chat/completions', 'model' => env('OPENROUTER_MODEL', 'meta-llama/llama-3.3-70b-instruct:free'), 'timeout' => 60, // batas waktu 60 detik 'max_jobs' => 20, // max 20 lowongan per request ke AI 'temperature' => 0.2, // rendah = jawaban AI lebih konsisten ];
```

13.3 File `bootstrap/app.php`

Tempat daftarin middleware alias biar bisa dipanggil pakai nama pendek di route:

```
return Application::configure(basePath: dirname(__DIR__)) ->withRouting( web: __DIR__.'/../routes/web.php', commands: __DIR__.'/../routes/console.php', health: '/up', ) ->withMiddleware(function (Middleware $middleware): void { // Daftarin nama alias buat middleware custom kita $middleware->alias([ 'auth.check' => \App\Http\Middleware\AuthCheck::class, 'admin.only' => \App\Http\Middleware\AdminOnly::class, 'recruiter.only' => \App\Http\Middleware\RecruiterOnly::class, ]); }) ->withExceptions(function (Exceptions $exceptions): void { // })->create();
```

Setelah didaftarkan di sini, kita bisa pakai `->middleware('admin.only')` di route, alih-alih nulis class path lengkap.

Dibuat: Juni 2026